

Funciones en Python

Una función es un bloque de código al que ponemos un nombre y que realiza una tarea específica. Uno de los principales motivos para emplear funciones es que permiten la **reutilización del código**. En lugar de tener que copiar y pegar el mismo código en diferentes partes de un programa, se puede crear una función y llamarla en diferentes lugares según sea necesario.

Permiten estructurar mejor nuestros programas, facilitando el mantenimiento y la comprensión del mismo.

Definición y Estructura de Funciones

Sintaxis Básica

En Python, una función se define con la palabra clave `def`, seguida del nombre de la función y paréntesis que pueden contener los parámetros de entrada.

Recomendaciones clave:

- Utiliza nombres descriptivos que reflejen su propósito
- Una función debe encargarse de una única tarea específica
- Facilita la comprensión y el mantenimiento del código

```
#definición de la función
def nombre_de_la_funcion(parametros):
    # cuerpo de la función
    return resultado


#llamada a la función
nombre_de_la_funcion()
```

La estructura clara y bien definida de las funciones es fundamental para crear código legible y mantenible. Dividir funciones complejas en funciones más pequeñas mejora significativamente la calidad del código.

En Python existe otra alternativa a la hora de crear funciones. El símbolo de flecha (`->`) **se utiliza para indicar el tipo de valor de retorno de una función, también conocido como anotación de tipo**. Es una forma de documentar el código para mejorar la legibilidad y la verificación estática (detección de errores de tipo en el código):

```
def saludar(nombre: str) -> str:
    return f"Hola, {nombre}"
```

Parámetros y Argumentos

Los parámetros permiten crear funciones flexibles y reutilizables. Podemos llamar a la misma función con diferentes argumentos según nuestras necesidades, lo que maximiza la reutilización del código.

Por ejemplo, imaginemos que tenemos una función que simplemente suma dos números. Podemos hacer que reciba dos parámetros y realice la suma. En este caso, a y b son parámetros de la función suma (podíamos haber cogido cualquier otro nombre para a y b, los nombres los elegimos nosotros).

```
def suma(a, b):
    print(a + b)

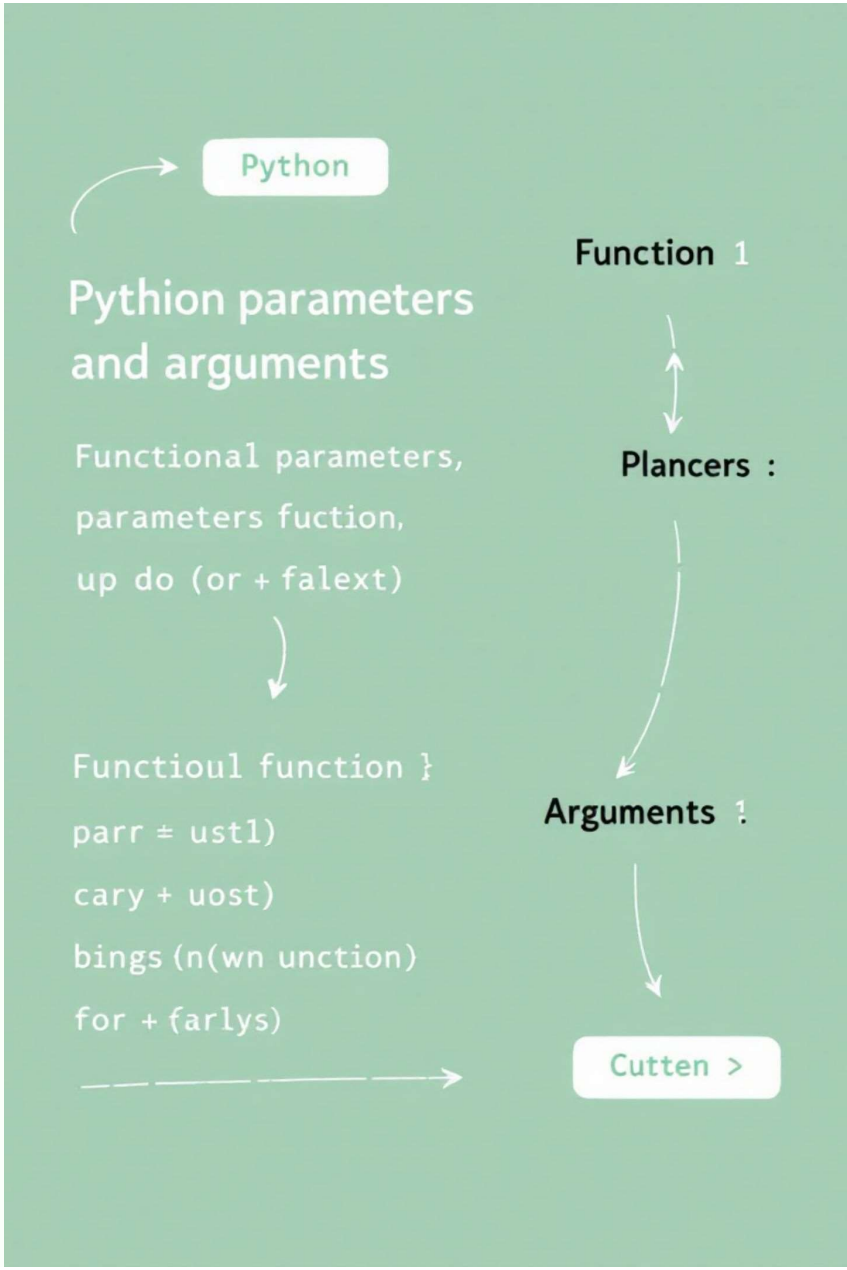
suma(3, 5) # llamamos a la función. Salida: 8
```

Parámetros

Son las variables que define la función en su declaración. Por ejemplo, en `suma(a, b)`, **a y b son parámetros**.

Argumentos

Son los valores concretos que pasamos a la función al invocarla. Por ejemplo, en `suma(3, 5)`, **3 y 5 son argumentos.**



Valores de Retorno

La palabra clave return

Las funciones pueden devolver valores utilizando `return`. Este valor puede ser asignado a una variable al llamar a la función.

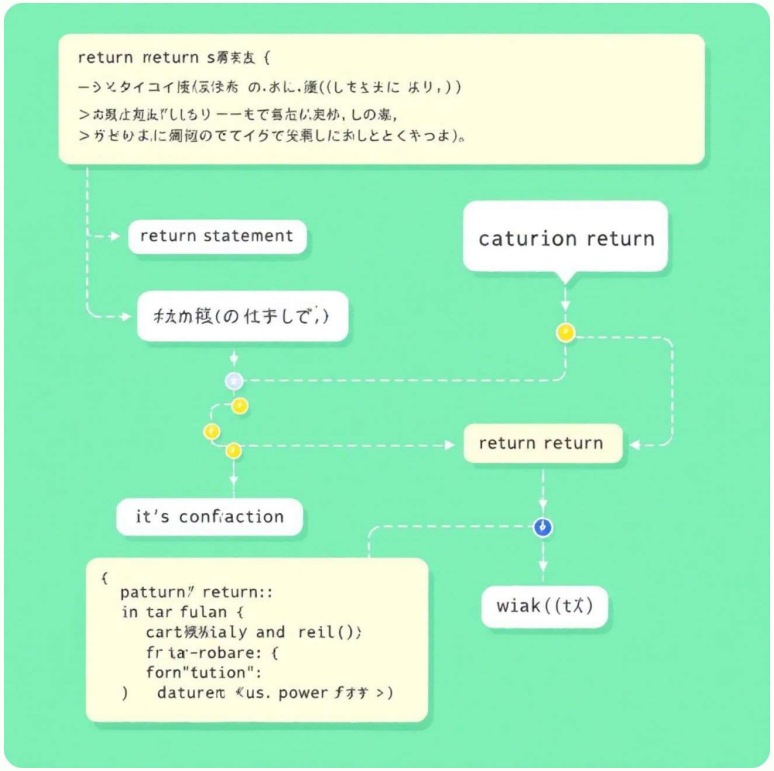
Ejemplo básico: Una función que suma dos números y devuelve el resultado:

```
def sumar(a, b):
    return a + b

resultado = sumar(4, 5)

print(resultado) # Muestra: 9
```

Cuando se alcanza un `return`, la función finaliza inmediatamente y devuelve el valor especificado al punto donde fue invocada.



Importante: Una función puede tener múltiples puntos de retorno, pero solo se ejecutará uno de ellos en cada llamada.

Paso de Parámetros: Mutables e Inmutables

Tipos Inmutables

Incluyen: números (enteros, reales, complejos, booleanos), cadenas y tuplas.

int(), float(), comp(), bool(), str(), tuple()

No permiten modificar el contenido sin destruir la referencia. Al modificar una variable inmutable, se crea un nuevo espacio en memoria.

Tipos Mutables

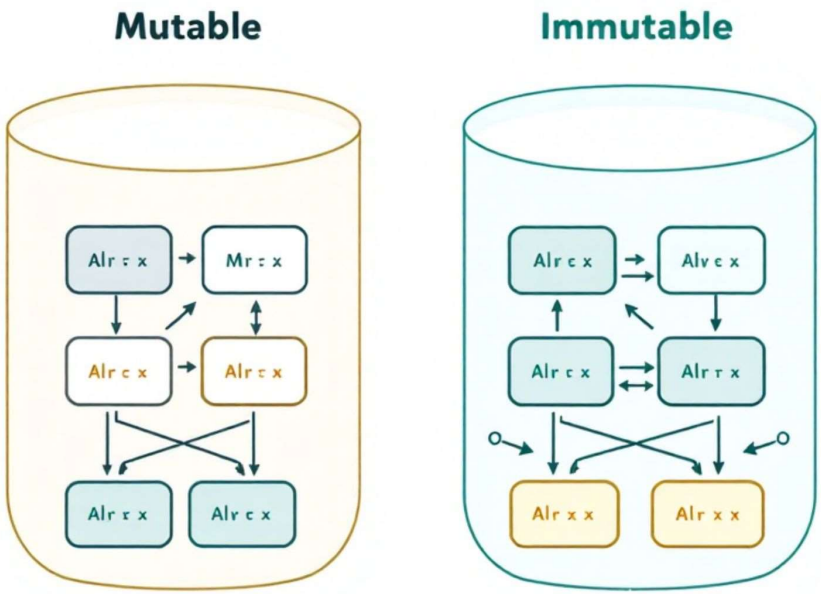
Incluyen: listas, diccionarios y conjuntos.

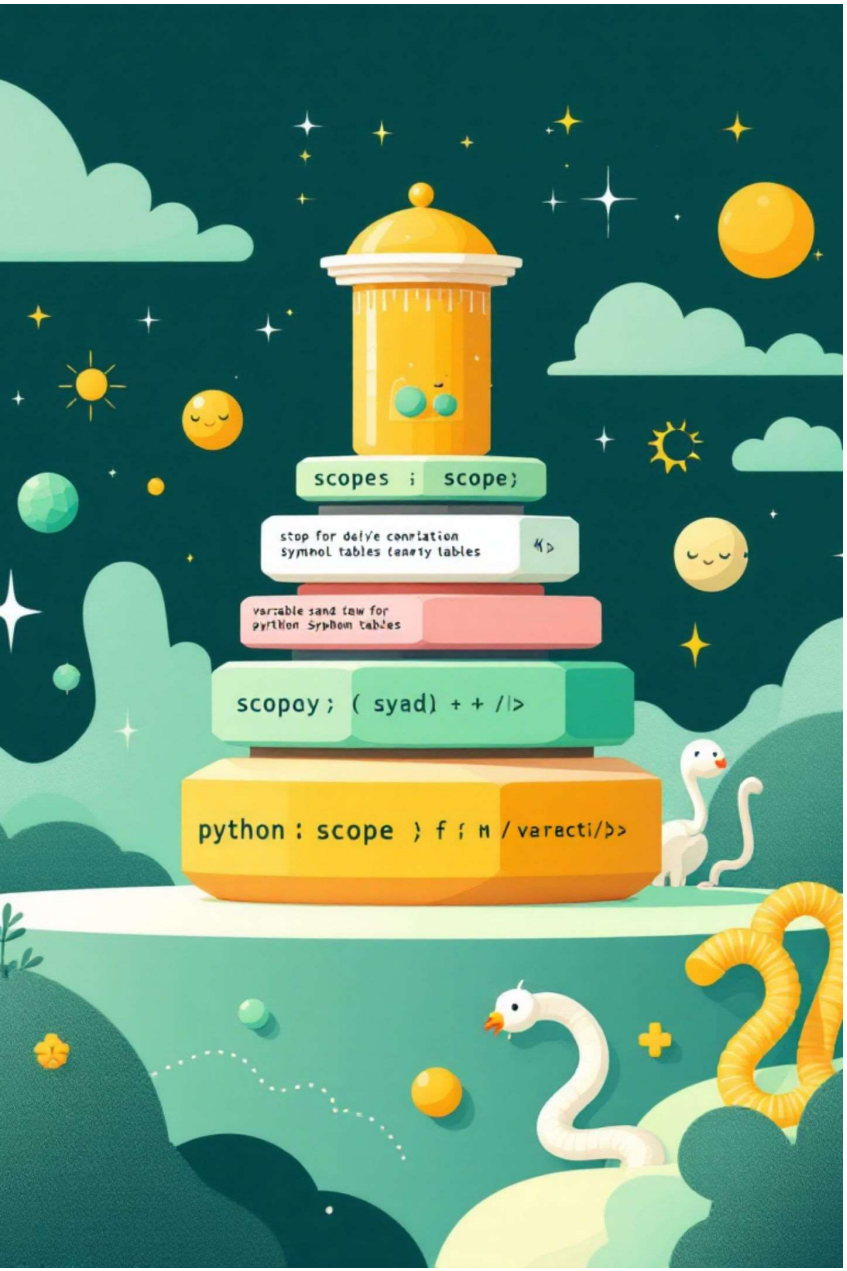
list(), dict(), set()

Permiten modificar el contenido sin cambiar la referencia. Las modificaciones dentro de una función afectan al objeto original.

¿?

Esta distinción es crucial: cuando pasamos objetos inmutables a una función, se comportan como paso por valor (se crea una copia). Los objetos mutables se comportan como paso por referencia (se modifica el original).





Ámbito y Tablas de Símbolos

¿Cómo funciona el mecanismo del ámbito y las tablas de símbolos?

Python mantiene unas tablas que, a modo de diccionario, relacionan los identificadores de los parámetros con las posiciones de memoria donde se guardan los datos. Estas tablas se denominan «**tablas de símbolos**».

Al entrar en una función, se crea una nueva tabla que se apila sobre las existentes. se crea una nueva tabla de símbolos y, por ende, un nuevo ámbito. Estas tablas de símbolos se van apilando (siguiendo una estructura de tipo «pila»), de tal manera que, al entrar en el ámbito de la función, se coloca una nueva tabla en lo alto de la pila. Cuando abandonamos el ámbito al salir de la función, la tabla en lo alto de la pila se borra y se libera la memoria asociada a parámetros y variables internas.

A la hora de buscar un identificador, al hacer uso de esa variable en nuestro código, se mira en la pila de tablas de símbolos, empezando por la más alta (el ámbito más interno) y bajando por ella (ámbitos externos).

Este mecanismo permite tener variables con el mismo nombre en diferentes ámbitos sin conflictos, manteniendo siempre la prioridad del ámbito más interno.

Creación del ámbito

Al llamar a una función, se crea una nueva tabla de símbolos en lo alto de la pila

Búsqueda de variables

Se busca desde el ámbito más interno (tabla superior) hacia los externos

Destrucción del ámbito

Al salir de la función, se elimina su tabla y se liberan sus variables

Pasando argumentos a funciones

(Lo vimos en el tema anterior)

Cuando pasamos variables a funciones, **debemos tener en cuenta si son tipos de valor o tipos de referencia**. En Python, los argumentos siempre se pasan por referencia, es decir, los identificadores definidos en la cabecera de la función (los parámetros) quedan referenciados a las posiciones de memoria de los valores indicados en la llamada a la función. Pero resulta que el objeto pasado como argumento puede ser mutable o inmutable.

Si es inmutable, los parámetros crean un nuevo espacio en memoria, lo cual sería equivalente a un paso por valor. Por tanto, cuando los argumentos son objetos inmutables su contenido no se ve alterado por la llamada a la función, pues los parámetros guardan su propia copia del argumento.

Pasando tipos de valor

Los tipos valor no se modifican cuando son pasados como parámetros, porque la función recibe una copia.

```
def duplicar_numero(num):
    num *= 2
    print("Dentro de la función:", num)
x = 10
duplicar_numero(x)
print("Fuera de la función:", x) #x sigue siendo 10, no se modificó
```

El valor de x no se modifica fuera de la función duplicar_numero porque es un tipo de valor.

En cambio, cuando pasamos como argumentos tipos mutables, la tabla de símbolos de la función también contiene sus propios identificadores, pero estos parámetros apuntan a las posiciones de memoria donde se alojan los datos pasados como argumento.

Pasando tipos de referencia

Los tipos referencia sí se modifican cuando son pasados como parámetros, porque la función recibe una copia de la referencia, que apunta a los mismos datos.

```
def modificar_lista(lista):
    lista.append("Nuevo elemento")
     print("Dentro de la función:", lista)
mi_lista = [1, 2, 3]
modificar_lista(mi_lista)
print("Fuera de la función:", mi_lista) # mi_lista se modifica
```

En este caso, mi_lista se modifica fuera de la función porque es un tipo de referencia.

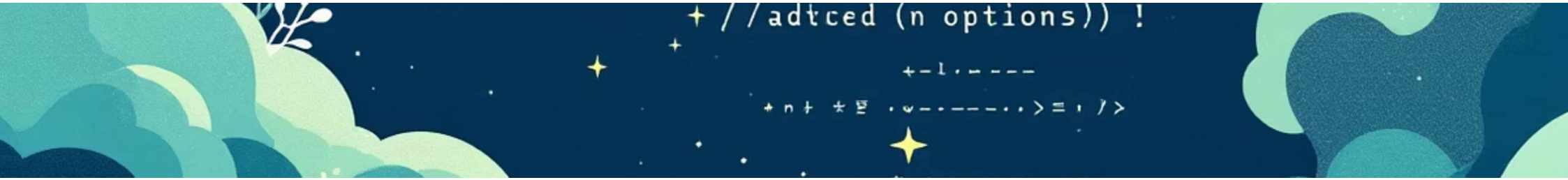
10

dulo = a */* b ## result

dulo = a // b: 3

or division 1 # a * b + 3

entiation # result: 1000



Opciones Avanzadas de Parámetros

Valores por Defecto

Podemos asignar valores por defecto a los parámetros en la definición de la función. Esto permite invocarla sin especificar todos los argumentos.

Ejemplo:

```
def saluda(nombre='amigo'):
    print('Hola', nombre)
saluda()  #'Hola amigo'
saluda('Antonio')  #'Hola Antonio'
```

Paso de valores por Clave

Sin especificar los argumentos, estamos usando el paso por posición.

Al especificar argumentos usando el nombre del parámetro, lo que mejora la legibilidad y permite cambiar el orden.

Ejemplo:

```
funcion(b=5, a=3)
```

Parámetros Indefinidos

Algunas funciones aceptan un número indeterminado de argumentos. Un ejemplo es la función print() que admite tantos argumentos como queramos, mostrando los valores pasados separados por espacios como una única cadena:

```
nombre = 'Charo'
edad = 15
print(nombre, 'tiene', edad, 'años') #Charo tiene 15 años
print('Hola', nombre) #Hola Charo
```

En Python, esto se denomina «lista de argumentos arbitraria» y permite usar tuplas o diccionarios a la hora de pasar los argumentos de una función. Para que nuestra función pueda ser invocada con un número indefinido de argumentos precedemos el parámetro con un asterisco (*) y dicho parámetro será interpretado como una tupla con los argumentos, o lo precedemos de doble asterisco (**) y el parámetro será interpretado como un diccionario que contendrá los pares <clave, valor> de un número arbitrario de argumentos pasados por clave.

```
def reserva(origen, destino, *viajeros, **detalles):
    print(f'Se ha reservado el vuelo de {origen} {destino} para:')
    for viajero in viajeros:
        print(viajero)
        print('Con los siguientes detalles:')
    for k in detalles:
        print(k, ':', detalles[k])
reserva('Granada', 'México', 'Pedro', 'Juani', 'Luisa', comida='vegana', asientos='ventanilla', precio=350)
```

La salida de este programa será:

```
Se ha reservado el vuelo de Granada México para:
Pedro
Con los siguientes detalles:
Juani
Con los siguientes detalles:
Luisa
Con los siguientes detalles:
comida : vegana
asientos : ventanilla
precio : 350
```

Salida de la función

Una función toma unos parámetros de entrada, realiza un procesamiento y finaliza su ejecución cuando llega a la última instrucción o cuando se alcanza un punto de retorno. Cuando cualquiera de estas dos situaciones se produce, se dice que «se sale de la función» y se regresa a la línea donde fue invocada.

El punto de retorno puede o no devolver un valor como resultado de la función. Cuando la función devuelve un valor, la llamada original de la función es reemplazada por el valor generado. Así, **print(suma(4, 5))** es reemplazado por **print(9)** antes de invocar a la función print().

Una función puede tener varios puntos de retorno:

```
def mayor(a, b):
    if a > b:
        return a
    return b
```



Recursividad

Una función recursiva es una función que se llama a sí misma. Un ejemplo típico sería el cálculo del factorial. El factorial de 4, escrito 4!, resulta de calcular el producto de 4 por el factorial de 3. A su vez, el factorial de 3 resulta de calcular el producto de 3 por el factorial de 2... y así sucesivamente hasta llegar a 0, cuyo factorial es 1. El resultado es 4 *3* 2 *1* 1. La implementación recursiva del factorial en Python sería como sigue:

```
def factorial(x):
    if x == 0:
        return 1
    return x * factorial(x-1)


print (factorial(4)) #24
```

Como puede observarse, en la función existe un «caso base» que no implica más recursividad. Toda función recursiva debe contemplar al menos un caso base, pues de lo contrario las llamadas recursivas podrían no acabar nunca.





Mejores Prácticas y Resumen

Diseño de Funciones

- Nombres descriptivos y claros
- Una única responsabilidad por función
- Parámetros bien definidos

Consideraciones

- Diferencia entre mutables e inmutables
- Gestión del ámbito de variables

Las funciones son fundamentales para estructurar código mantenible y reutilizable.